

RV32IMB instruction cheat sheet

RV32I base + M (multiply/divide) + B (= Zba + Zbb + Zbs bit manipulation). All instructions are 32 bits. **sext** = sign-extend, **zext** = zero-extend, all registers are 32 bits. Immediates in examples are decimal unless written 0x...; branch and jump offsets are byte offsets relative to the instruction itself. The **signed int32** column is the machine word read as a two's-complement number (what a Factorio signal carries).

Field colours: opcode funct3 / funct7 / fixed register immediate

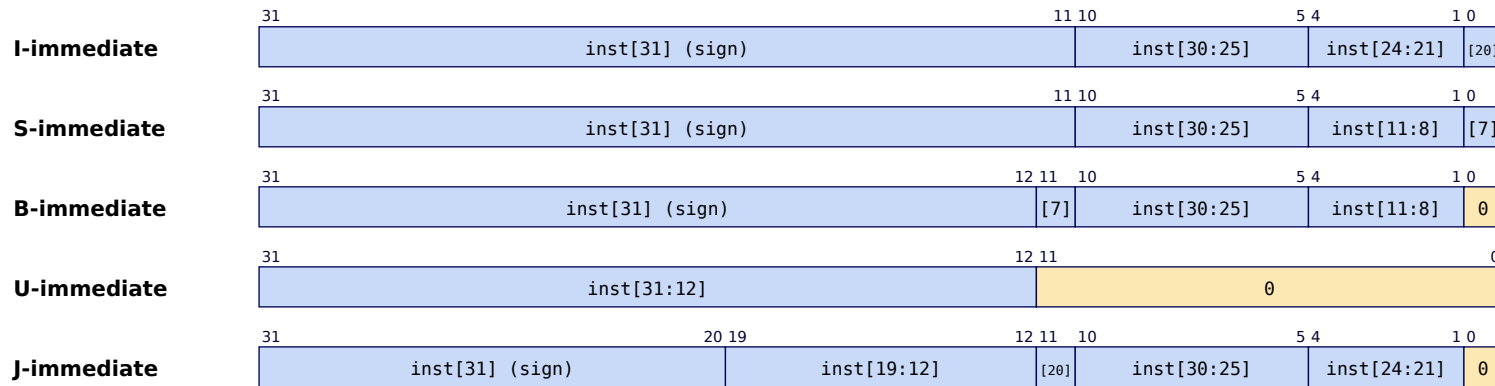
Base instruction formats (Sec. 2.2 / 2.3)

R-type	312524201915141211760 funct7rs2rs1funct3rdopcode	register-register (OP), incl. M, Zba, most of Zbb/Zbs
I-type	31201915141211760 imm[11:0]rs1funct3rdopcode	OP-IMM, loads, JALR
S-type	312524201915141211760 imm[11:5]rs2rs1funct3imm[4:0]opcode	stores
B-type	313025242019151412118760 [12]imm[10:5]rs2rs1funct3imm[4:1][11]opcode	branches, offset bit 0 is always 0
U-type	311211760 imm[31:12]rdopcode	LUI, AUIPC
J-type	31302120191211760 [20]imm[10:1][11]imm[19:12]rdopcode	JAL, offset bit 0 is always 0

Format variants used by specific instructions

I (shamt)	312524201915141211760 funct7shamt rs1funct3rdopcode	slli srli srli, rori, bclri bexti binvi bseti
I (unary)	312524201915141211760 funct7funct5 rs1funct3rdopcode	clz ctz cpop sext.b sext.h (OP-IMM)
I (unary 12)	31201915141211760 funct12 rs1funct3rdopcode	orc.b, rev8 (OP-IMM); ecall, ebreak (SYSTEM)
R (unary)	312524201915141211760 funct7rs2=0 rs1funct3rdopcode	zext.h (OP)
I (fence)	3128272423201915141211760 fm pred succ rs1funct3rdopcode	fence, fence.tso, pause; pred/succ bits = I O R W

Immediate values produced by each format (Figure 1): bit positions of the 32-bit immediate, labelled with the instruction bits that fill them



The sign bit of every immediate is always `inst[31]`. Ranges: I/S –2048...2047 · B –4096...4094 (even) · J –1048576...1048574 (even) · U: `imm20` 0...0xFFFFF (value = `imm20` × 4096) · `shamt` 0...31.

Major opcodes used (Table 72, `inst[1:0] = 11`)

Opcode	<code>inst[6:0]</code>	Hex	Dec	Formats	Instructions
LOAD	0000011	0x03	3	I	<code>lb lh lw lbu lhu</code>
MISC-MEM	0001111	0x0F	15	I (fence)	<code>fence fence.tso pause</code>
OP-IMM	0010011	0x13	19	I, I (shamt), I (unary)	<code>addi slti sltiu xori ori andi slli srli srai clz ctz cpop sext.b sext.h rori orc.b rev8 bclri bexti binvi bseti</code>
AUIPC	0010111	0x17	23	U	<code>auipc</code>
STORE	0100011	0x23	35	S	<code>sb sh sw</code>
OP	0110011	0x33	51	R, R (unary)	<code>add sub sll slt sltu xor srl sra or and mul mulh mulhsu mulhu div divu rem remu sh1add sh2add sh3add andn orn xnor max maxu min minu zext.h rol ror bclr bext binv bset</code>
LUI	0110111	0x37	55	U	<code>lui</code>
BRANCH	1100011	0x63	99	B	<code>beq bne blt bge bltu bgeu</code>
JALR	1100111	0x67	103	I	<code>jalr</code>
JAL	1101111	0x6F	111	J	<code>jal</code>
SYSTEM	1110011	0x73	115	I (system)	<code>ecall ebREAK</code>

Registers (x0 is hard-wired to 0; ABI names accepted by the tools)

Reg	ABI	Reg	ABI	Reg	ABI	Reg	ABI
x0	zero	x8	s0 / fp	x16	a6	x24	s8
x1	ra	x9	s1	x17	a7	x25	s9
x2	sp	x10	a0	x18	s2	x26	s10
x3	gp	x11	a1	x19	s3	x27	s11
x4	tp	x12	a2	x20	s4	x28	t3
x5	t0	x13	a3	x21	s5	x29	t4
x6	t1	x14	a4	x22	s6	x30	t5
x7	t2	x15	a5	x23	s7	x31	t6

Instructions

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: integer register-immediate								
addi	rd, rs1, imm	I	OP-IMM 0010011	000	–	rd = rs1 + sext(imm). Overflow is ignored.	addi x1, x2, 23	0x01710093 24182931
slti	rd, rs1, imm	I	OP-IMM 0010011	010	–	rd = (rs1 < sext(imm)) ? 1 : 0, signed compare	slti x5, x6, -1	0xFFFF32293 -843117
sltiu	rd, rs1, imm	I	OP-IMM 0010011	011	–	rd = (rs1 < sext(imm)) ? 1 : 0, unsigned compare (imm is sign-extended first, then treated as unsigned)	sltiu x5, x6, 1	0x00133293 1258131
xori	rd, rs1, imm	I	OP-IMM 0010011	100	–	rd = rs1 ^ sext(imm)	xori x5, x6, -1	0xFFFF34293 -834925
ori	rd, rs1, imm	I	OP-IMM 0010011	110	–	rd = rs1 sext(imm)	ori x5, x6, 0x0F0	0x0F036293 251880083
andi	rd, rs1, imm	I	OP-IMM 0010011	111	–	rd = rs1 & sext(imm)	andi x5, x6, 255	0x0FF37293 267612819
slli	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0000000	rd = rs1 << shamt (logical left shift, shamt 0..31)	slli x5, x6, 4	0x00431293 4395667
srli	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0000000	rd = rs1 >> shamt, logical (zeros shifted in)	srli x5, x6, 4	0x00435293 4412051
srai	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0100000	rd = rs1 >> shamt, arithmetic (sign bit copied in)	srai x5, x6, 4	0x40435293 1078153875
Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: integer register-register								
add	rd, rs1, rs2	R	OP 0110011	000	0000000	rd = rs1 + rs2. Overflow is ignored.	add x3, x4, x5	0x005201B3 5374387
sub	rd, rs1, rs2	R	OP 0110011	000	0100000	rd = rs1 – rs2	sub x3, x4, x5	0x405201B3 1079116211
sll	rd, rs1, rs2	R	OP 0110011	001	0000000	rd = rs1 << rs2[4:0] (logical left shift)	sll x3, x4, x5	0x005211B3 5378483
slt	rd, rs1, rs2	R	OP 0110011	010	0000000	rd = (rs1 < rs2) ? 1 : 0, signed	slt x3, x4, x5	0x005221B3 5382579
sltu	rd, rs1, rs2	R	OP 0110011	011	0000000	rd = (rs1 < rs2) ? 1 : 0, unsigned	sltu x3, x4, x5	0x005231B3 5386675
xor	rd, rs1, rs2	R	OP 0110011	100	0000000	rd = rs1 ^ rs2	xor x3, x4, x5	0x005241B3 5390771
srl	rd, rs1, rs2	R	OP 0110011	101	0000000	rd = rs1 >> rs2[4:0], logical	srl x3, x4, x5	0x005251B3 5394867
sra	rd, rs1, rs2	R	OP 0110011	101	0100000	rd = rs1 >> rs2[4:0], arithmetic	sra x3, x4, x5	0x405251B3 1079136691
or	rd, rs1, rs2	R	OP 0110011	110	0000000	rd = rs1 rs2	or x3, x4, x5	0x005261B3 5398963
and	rd, rs1, rs2	R	OP 0110011	111	0000000	rd = rs1 & rs2	and x3, x4, x5	0x005271B3 5403059

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: upper immediates								
lui	rd, imm20	U	LUI 0110111	–	–	rd = imm20 << 12 (lower 12 bits zero). Builds 32-bit constants (imm20: 0..0xFFFFF).	lui x5, 0x12345	0x123452B7 305418935
auipc	rd, imm20	U	AUIPC 0010111	–	–	rd = pc + (imm20 << 12). Builds pc-relative addresses.	auipc x5, 0x10	0x00010297 66199
Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: control transfer (offsets in bytes, relative to this instruction)								
jal	rd, offset	J	JAL 1101111	–	–	rd = pc + 4; pc = pc + offset. Offset is even, ±1 MiB.	jal ra, 2048	0x001000EF 1048815
jalr	rd, offset(rs1)	I	JALR 1100111	000	–	t = pc + 4; pc = (rs1 + sext(offset)) & ~1; rd = t	jalr x1, 12(x5)	0x00C280E7 12746983
beq	rs1, rs2, offset	B	BRANCH 1100011	000	–	if rs1 == rs2: pc = pc + offset. Offset is even, ±4 KiB.	beq x1, x2, -8	0xFE208CE3 -31421213
bne	rs1, rs2, offset	B	BRANCH 1100011	001	–	if rs1 ≠ rs2: pc = pc + offset	bne x1, x2, 16	0x00209863 2136163
blt	rs1, rs2, offset	B	BRANCH 1100011	100	–	if rs1 < rs2 (signed): pc = pc + offset	blt x1, x2, -8	0xFE20CCE3 -31404829
bge	rs1, rs2, offset	B	BRANCH 1100011	101	–	if rs1 ≥ rs2 (signed): pc = pc + offset	bge x1, x2, 8	0x0020D463 2151523
bltu	rs1, rs2, offset	B	BRANCH 1100011	110	–	if rs1 < rs2 (unsigned): pc = pc + offset	bltu x1, x2, -4096	0x8020E063 -2145329053
bgeu	rs1, rs2, offset	B	BRANCH 1100011	111	–	if rs1 ≥ rs2 (unsigned): pc = pc + offset	bgeu x1, x2, 4094	0x7E20FFE3 2116091875
Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: loads and stores (address = rs1 + sext(offset))								
lb	rd, offset(rs1)	I	LOAD 0000011	000	–	rd = sext(mem8[addr]), 8-bit value sign-extended	lb x5, 0(x6)	0x00030283 197251
lh	rd, offset(rs1)	I	LOAD 0000011	001	–	rd = sext(mem16[addr]), 16-bit value sign-extended	lh x5, 2(x6)	0x00231283 2298499
lw	rd, offset(rs1)	I	LOAD 0000011	010	–	rd = mem32[addr]	lw a0, -4(sp)	0xFFC12503 -4119293
lbu	rd, offset(rs1)	I	LOAD 0000011	100	–	rd = zext(mem8[addr]), 8-bit value zero-extended	lbu x5, 1(x6)	0x00134283 1262211
lhu	rd, offset(rs1)	I	LOAD 0000011	101	–	rd = zext(mem16[addr]), 16-bit value zero-extended	lhu x5, 6(x6)	0x00635283 6509187
sb	rs2, offset(rs1)	S	STORE 0100011	000	–	mem8[addr] = rs2[7:0]	sb x5, 3(x6)	0x005301A3 5439907
sh	rs2, offset(rs1)	S	STORE 0100011	001	–	mem16[addr] = rs2[15:0]	sh x5, 2(x6)	0x00531123 5443875
sw	rs2, offset(rs1)	S	STORE 0100011	010	–	mem32[addr] = rs2	sw x5, 2047(x6)	0x7E532FA3 2119380899

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
RV32I: memory ordering and environment								
fence	pred, succ	I (fence)	MISC-MEM 0001111	000	–	Order device I/O and memory accesses: no other hart or device sees an operation in the successor set before any in the predecessor set. Sets are letters from i, o, r, w.	fence rw, w	0x0310000F 51380239
fence.tso	–	I (fence)	MISC-MEM 0001111	000	fm 1000 pred 0011 succ 0011	FENCE with fm=1000, pred=succ=rw: loads ordered before all later accesses, stores before later stores (total store ordering).	fence.tso	0x8330000F -2094006257
pause	–	I (fence)	MISC-MEM 0001111	000	fm 0000 pred 0001 succ 0000	Zihintpause hint: temporarily reduce the instruction retirement rate (e.g. in spin-wait loops). Encoded as FENCE with pred=w, succ=0.	pause	0x0100000F 16777231
ecall	–	I (system)	SYSTEM 1110011	000	000000000000	Environment call: service request to the execution environment (trap).	ecall	0x00000073 115
ebreak	–	I (system)	SYSTEM 1110011	000	000000000001	Breakpoint: return control to a debugging environment (trap).	ebreak	0x00100073 1048691
Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
M: multiplication and division								
mul	rd, rs1, rs2	R	OP 0110011	000	0000001	rd = (rs1 × rs2)[31:0], lower 32 bits of the product	mul x1, x2, x3	0x023100B3 36765875
mulh	rd, rs1, rs2	R	OP 0110011	001	0000001	rd = (rs1 × rs2)[63:32], signed × signed	mulh x1, x2, x3	0x023110B3 36769971
mulhsu	rd, rs1, rs2	R	OP 0110011	010	0000001	rd = (rs1 × rs2)[63:32], signed rs1 × unsigned rs2	mulhsu x1, x2, x3	0x023120B3 36774067
mulhu	rd, rs1, rs2	R	OP 0110011	011	0000001	rd = (rs1 × rs2)[63:32], unsigned × unsigned	mulhu x1, x2, x3	0x023130B3 36778163
div	rd, rs1, rs2	R	OP 0110011	100	0000001	rd = rs1 / rs2, signed, rounds towards zero. x/0 = −1; $-2^{31} / -1 = -2^{31}$.	div x1, x2, x3	0x023140B3 36782259
divu	rd, rs1, rs2	R	OP 0110011	101	0000001	rd = rs1 / rs2, unsigned. x/0 = $2^{32} - 1$ (all ones).	divu x1, x2, x3	0x023150B3 36786355
rem	rd, rs1, rs2	R	OP 0110011	110	0000001	rd = rs1 % rs2, signed; a nonzero result has the sign of the dividend. x % 0 = x; $-2^{31} \% -1 = 0$.	rem x1, x2, x3	0x023160B3 36790451
remu	rd, rs1, rs2	R	OP 0110011	111	0000001	rd = rs1 % rs2, unsigned. x % 0 = x.	remu x31, x30, x29	0x03DF7FB3 64978867
Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zba: address generation								
shladd	rd, rs1, rs2	R	OP 0110011	010	0010000	rd = rs2 + (rs1 << 1)	shladd a0, a1, a2	0x20C5A533 549823795
sh2add	rd, rs1, rs2	R	OP 0110011	100	0010000	rd = rs2 + (rs1 << 2)	sh2add a0, a1, a2	0x20C5C533 549831987
sh3add	rd, rs1, rs2	R	OP 0110011	110	0010000	rd = rs2 + (rs1 << 3)	sh3add a0, a1, a2	0x20C5E533 549840179

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zbb: basic bit manipulation								
andn	rd, rs1, rs2	R	OP 0110011	111	0100000	rd = rs1 & ~rs2	andn x1, x2, x3	0x403170B3 1076981939
orn	rd, rs1, rs2	R	OP 0110011	110	0100000	rd = rs1 ~rs2	orn x1, x2, x3	0x403160B3 1076977843
xnor	rd, rs1, rs2	R	OP 0110011	100	0100000	rd = ~(rs1 ^ rs2)	xnor x1, x2, x3	0x403140B3 1076969651
clz	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00000	rd = number of leading zero bits, counted from bit 31 (32 if rs1 = 0)	clz x3, x4	0x60021193 1610748307
ctz	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00001	rd = number of trailing zero bits, counted from bit 0 (32 if rs1 = 0)	ctz x3, x4	0x60121193 1611796883
cpop	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00010	rd = number of 1 bits in rs1 (population count)	cpop x1, x2	0x60211093 1612779667
max	rd, rs1, rs2	R	OP 0110011	110	0000101	rd = larger of rs1, rs2 (signed)	max x1, x2, x3	0x0A3160B3 171008179
maxu	rd, rs1, rs2	R	OP 0110011	111	0000101	rd = larger of rs1, rs2 (unsigned)	maxu x1, x2, x3	0x0A3170B3 171012275
min	rd, rs1, rs2	R	OP 0110011	100	0000101	rd = smaller of rs1, rs2 (signed)	min x1, x2, x3	0x0A3140B3 170999987
minu	rd, rs1, rs2	R	OP 0110011	101	0000101	rd = smaller of rs1, rs2 (unsigned)	minu x1, x2, x3	0x0A3150B3 171004083
sext.b	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00100	rd = sext(rs1[7:0]): bit 7 copied into bits 31..8	sext.b x1, x2	0x60411093 1614876819
sext.h	rd, rs1	I (unary)	OP-IMM 0010011	001	0110000 00101	rd = sext(rs1[15:0]): bit 15 copied into bits 31..16	sext.h x1, x2	0x60511093 1615925395
zext.h	rd, rs1	R (unary)	OP 0110011	100	0000100 00000	rd = zext(rs1[15:0]): bits 31..16 cleared	zext.h x1, x2	0x080140B3 134299827
rol	rd, rs1, rs2	R	OP 0110011	001	0110000	rd = rs1 rotated left by rs2[4:0]	rol x1, x2, x3	0x603110B3 1613828275
ror	rd, rs1, rs2	R	OP 0110011	101	0110000	rd = rs1 rotated right by rs2[4:0]	ror x1, x2, x3	0x603150B3 1613844659
rori	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0110000	rd = rs1 rotated right by shamt (0..31)	rori x1, x2, 7	0x60715093 1618038931
orc.b	rd, rs1	I (unary)	OP-IMM 0010011	101	001010000111	OR-combine per byte: each byte of rd is 0x00 if that byte of rs1 is 0, else 0xFF	orc.b x1, x2	0x28715093 678514835
rev8	rd, rs1	I (unary)	OP-IMM 0010011	101	011010011000	rd = rs1 with the byte order reversed	rev8 x1, x2	0x69815093 1770082451

Mnemonic	Arguments	Format	Opcode	funct3	funct7 / fixed	Description	Example	Hex · signed int32
B / Zbs: single-bit instructions (bit index = rs2[4:0] or shamt 0..31)								
bclr	rd, rs1, rs2	R	OP 0110011	001	0100100	rd = rs1 & ~(1 << rs2[4:0]) (clear bit)	bclr x1, x2, x3	0x483110B3 1211175091
bclri	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0100100	rd = rs1 & ~(1 << shamt)	bclri x1, x2, 5	0x48511093 1213272211
bext	rd, rs1, rs2	R	OP 0110011	101	0100100	rd = (rs1 >> rs2[4:0]) & 1 (extract bit)	bext x1, x2, x3	0x483150B3 1211191475
bexti	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	101	0100100	rd = (rs1 >> shamt) & 1	bexti x1, x2, 5	0x48515093 1213288595
binv	rd, rs1, rs2	R	OP 0110011	001	0110100	rd = rs1 ^ (1 << rs2[4:0]) (invert bit)	binv x1, x2, x3	0x683110B3 1748046003
binvi	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0110100	rd = rs1 ^ (1 << shamt)	binvi x1, x2, 5	0x68511093 1750143123
bset	rd, rs1, rs2	R	OP 0110011	001	0010100	rd = rs1 (1 << rs2[4:0]) (set bit)	bset x1, x2, x3	0x283110B3 674304179
bseti	rd, rs1, shamt	I (shamt)	OP-IMM 0010011	001	0010100	rd = rs1 (1 << shamt)	bseti x1, x2, 31	0x29F11093 703664275

Pseudoinstructions (only those defined in the spec)

Pseudo	Assembled as	Meaning	Example	Example assembled as	Hex · signed int32
nop	addi x0, x0, 0	no operation	nop	addi x0, x0, 0	0x00000013 · 19
mv rd, rs	addi rd, rs, 0	copy register	mv a0, a1	addi x10, x11, 0	0x00058513 · 361747
not rd, rs	xori rd, rs, -1	bitwise NOT	not a0, a1	xori x10, x11, -1	0xFFFF5C513 · -670445
seqz rd, rs	sltiu rd, rs, 1	rd = (rs == 0) ? 1 : 0	seqz a0, a1	sltiu x10, x11, 1	0x0015B513 · 1422611
snez rd, rs	sltu rd, x0, rs	rd = (rs ≠ 0) ? 1 : 0	snez a0, a1	sltu x10, x0, x11	0x00B03533 · 11547955
j offset	jal x0, offset	jump, no link	j -16	jal x0, -16	0xFF1FF06F · -14684049
jr rs	jalr x0, 0(rs)	jump to register, no link	jr t0	jalr x0, 0(x5)	0x00028067 · 163943
ret	jalr x0, 0(x1)	return from subroutine (to address in ra)	ret	jalr x0, 0(x1)	0x00008067 · 32871
bgt rs, rt, offset	blt rt, rs, offset	branch if rs > rt (signed): operands swapped	bgt a0, a1, 8	blt x11, x10, 8	0x00A5C463 · 10863715
bgtu rs, rt, offset	bltu rt, rs, offset	branch if rs > rt (unsigned)	bgtu a0, a1, 8	bltu x11, x10, 8	0x00A5E463 · 10871907
ble rs, rt, offset	bge rt, rs, offset	branch if rs ≤ rt (signed)	ble a0, a1, 8	bge x11, x10, 8	0x00A5D463 · 10867811
bleu rs, rt, offset	bgeu rt, rs, offset	branch if rs ≤ rt (unsigned)	bleu a0, a1, 8	bgeu x11, x10, 8	0x00A5F463 · 10876003

Operand short forms accepted by the tools

Short form	Same as	Meaning	Example	Example assembled as	Hex · signed int32
jal offset	jal x1, offset	call: link register ra	jal 256	jal x1, 256	0x100000EF · 268435695
jalr rs	jalr x1, 0(rs)	call through register	jalr t0	jalr x1, 0(x5)	0x000280E7 · 164071
jalr rd, rs1, offset	jalr rd, offset(rs1)	alternative operand order	jalr x1, x5, 12	jalr x1, 12(x5)	0x00C280E7 · 12746983
lw rd, (rs1)	lw rd, 0(rs1)	offset may be omitted (all loads/stores)	lw a0, (sp)	lw x10, 0(x2)	0x00012503 · 75011
fence	fence iorw, iorw	full fence	fence	fence iorw, iorw	0x0FF0000F · 267386895